Problem List

Submit

Premium

Description | Editorial | Solutions | Submissions

125. Valid Palindrome

Solved

Easy | Topics | Companies

A phrase is a **palindrome** if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Given a string `s`, return `true` *if it is a* ***palindrome****, or* `false` *otherwise.*

Example 1:

```
Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.
```

Example 2:

```
Input: s = "race a car"
Output: false
Explanation: "raceacar" is not a palindrome.
```

Example 3:

```
Input: s = " "
Output: true
Explanation: s is an empty string "" after removing non-alphanumeric characters.
Since an empty string reads the same forward and backward, it is a palindrome.
```

11.4K | 428 | 378 Online

Code

Java | Auto

```java
class Solution {
    public boolean isPalindrome(String s) {
        int l = 0;
        int r = s.length() - 1;

        while (l < r) {
            while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
            while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;

            if (Character.toLowerCase(s.charAt(l)) !=
                Character.toLowerCase(s.charAt(r))) {
                return false;
            }

            l++;
            r--;
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

s =
"A man, a plan, a canal: Panama"

Output

true

Description | Editorial | Solutions | Submissions

557. Reverse Words in a String III

Solved

Easy | Topics | Companies

Given a string `s`, reverse the order of characters in each word within a sentence while still preserving whitespace and initial word order.

Example 1:

```
Input: s = "Let's take LeetCode contest"
Output: "s'teL ekat edoCteeL tsetnoc"
```

Example 2:

```
Input: s = "Mr Ding"
Output: "rM gniD"
```

Constraints:

- $1 <= s.length <= 5 * 10^4$
- `s` contains printable **ASCII** characters.
- `s` does not contain any leading or trailing spaces.
- There is **at least one** word in `s`.
- All the words in `s` are separated by a single space.

6.2K 106 27 Online

Code

Java | Auto

```java
class Solution {
    public String reverseWords(String s) {

        char[] arr = s.toCharArray();
        int left = 0;

        for (int i = 0; i <= arr.length; i++) {

            if (i == arr.length || arr[i] == ' ') {

                int l = left, r = i - 1;

                while (l < r) {
                    char temp = arr[l];
                    arr[l] = arr[r];
                    arr[r] = temp;
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2

Input

s =
"Let's take LeetCode contest"

Output

"s'teL ekat edoCteeL tsetnoc"

Description | Editorial | Solutions | Submissions

387. First Unique Character in a String

Solved

Easy | Topics | Companies

Given a string `s`, find the **first** non-repeating character in it and return its index. If it **does not** exist, return `-1`.

Example 1:

> **Input:** `s = "leetcode"`
>
> **Output:** `0`
>
> **Explanation:**
>
> The character `'l'` at index 0 is the first character that does not occur at any other index.

Example 2:

> **Input:** `s = "loveleetcode"`
>
> **Output:** 2

Example 3:

> **Input:** `s = "aabb"`
>
> **Output:** -1

Code

Java | Auto

```java
class Solution {
    public int firstUniqChar(String s) {
        int[] freq = new int[26];
        for (int i = 0; i < s.length(); i++) {
            freq[s.charAt(i) - 'a']++;
        }
        for (int i = 0; i < s.length(); i++) {
            if (freq[s.charAt(i) - 'a'] == 1) {
                return i;
            }
        }
        return -1;
    }
}
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

s =
"leetcode"

Output

0

Problem List

Submit

Premium

Description | Editorial | Solutions | Submissions

424. Longest Repeating Character Replacement

Solved

Medium | Topics | Companies

You are given a string `s` and an integer `k`. You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most `k` times.

Return *the length of the longest substring containing the same letter you can get after performing the above operations.*

Example 1:

```
Input: s = "ABAB", k = 2
Output: 4
Explanation: Replace the two 'A's with two 'B's or vice versa.
```

Example 2:

```
Input: s = "AABABBA", k = 1
Output: 4
Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA".
The substring "BBBB" has the longest repeating letters, which is 4.
There may exists other ways to achieve this answer too.
```

12.8K 331 368 Online

Code Run Ctrl

Java Auto

```java
class Solution {
    public int characterReplacement(String s, int k) {

        int count[] = new int[26];
        int l =0, maxf =0,res=0;

        for(int r =0; r<s.length();r++){
            int rf = s.charAt(r)-'A';
            int lf = s.charAt(l)-'A';
            count[rf]++;
            maxf = Math.max(maxf, count[rf]);
            while((r-l+1)-maxf>k){
                count[lf]--;
                l++;
            }
            res = Math.max(res,r-l+1);
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

s =
"ABAB"

k =
2

Problem List

Submit

Premium

Description | Editorial | Solutions | Submissions

76. Minimum Window Substring

Solved

Hard | Topics | Companies | Hint

Given two strings `s` and `t` of lengths `m` and `n` respectively, return *the* ***minimum window*** *substring of* `s` *such that every character in* `t` *(**including duplicates**) is included in the window.* If there is no such substring, return *the empty string* `""`.

The testcases will be generated such that the answer is **unique**.

Example 1:

```
Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
Explanation: The minimum window substring "BANC" includes
'A', 'B', and 'C' from string t.
```

Example 2:

```
Input: s = "a", t = "a"
Output: "a"
Explanation: The entire string s is the minimum window.
```

Example 3:

```
Input: s = "a", t = "aa"
Output: ""
Explanation: Both 'a's from t must be included in the
window.
Since the largest window of s only has one 'a', return
```

19.9K | 329 | 381 Online

Code | Run Ctrl '

Java | Auto

```java
import java.util.*;

class Solution {
    public String minWindow(String s, String t) {
        if (t.length() == 0) return "";
        Map<Character, Integer> needMap = new HashMap<>();
        for (char c : t.toCharArray()) {
            needMap.put(c, needMap.getOrDefault(c, 0) + 1);
        }

        Map<Character, Integer> winMap = new HashMap<>();

        int have = 0;
        int need = needMap.size();

        int l = 0;
```

Saved

Ln 1, Col 1

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 | Case 2 | Case 3

Input

s =
"ADOBECODEBANC"

t =
"ABC"